



FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA - UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Fleet Manager

André Luiz Cavalcante da Silva (2310287)

Luiz Eduardo Pacheco (2310314)

Gregory Figueiredo de M P Jereissati (2320425)

Fortaleza-CE

2026

André Luiz Cavalcante da Silva (2310287)

Luiz Eduardo Pacheco (2310314)

Gregory Figueiredo de M P Jereissati (2320425)

Fleet Manager

Trabalho de Conclusão do Curso apresentado ao curso do CENTRO DE CIÊNCIAS TECNOLÓGICAS da Universidade de Fortaleza, para obtenção do grau de Bacharel em CIÊNCIA DA COMPUTAÇÃO.

Orientador: Prof. Me. Ronnison Reges Vidal

Fortaleza-CE

2026

RESUMO

O *Fleet Manager* é um sistema web desenvolvido para automatizar e otimizar a gestão de frotas veiculares, centralizando o controle de veículos, motoristas, despesas, manutenções e alertas de documentação. A arquitetura do sistema foi concebida sob o padrão de *Monorepo*, gerenciado pela ferramenta Turborepo, promovendo alto reuso de código. A camada de interface (*Frontend*) foi desenvolvida em React.js utilizando Vite e estilização via TailwindCSS, enquanto a camada de serviços (*API Backend*) foi construída em Node.js com Express. Para a persistência, o projeto adota um banco de dados relacional PostgreSQL totalmente gerenciado pela plataforma Supabase, acessado de forma unificada através do Prisma ORM. A segurança e o isolamento de dados são garantidos por uma solução própria de Autenticação (Custom Auth) gerando *JSON Web Tokens (JWT)* com criptografia, aliada a um rigoroso Controle de Acesso Baseado em Papéis (RBAC) que divide permissões entre Administrador, Gestor e Operador. O sistema é hospedado em nuvem de forma apartada, com o *Frontend* distribuído via Vercel, entregando uma solução escalável, de fácil manutenção e com alta disponibilidade para auxiliar na tomada de decisão estratégica e redução de custos logísticos.

Palavras-chave: Gestão de Frotas; Arquitetura de Software; Monorepo; Aplicações Web; Controle de Acesso (RBAC). .

LISTA DE ILUSTRAÇÕES

- **Figura 1** – Arquitetura 4+1
- **Figura 1** – Exemplo de Diagrama (com os casos de uso significativos e atores)
- **Figura 2** – Exemplo de Diagrama de Camadas da Aplicação
- **Figura 3** – Exemplo de Diagrama de Pacotes da Aplicação
- **Figura 20** – Exemplo de Diagrama de Classes
- **Figura 20** – Exemplo de Diagrama de Sequência

LISTA DE TABELAS

- **Tabela 1** – Visões, Público, Área e Artefatos da MDS
- **Tabela 2** – Exemplo de requisitos e restrições
- **Tabela 3** – Cronograma Resumido
- **Tabela 4** – Partes interessadas do projeto (Stakeholders)
- **Tabela 5** – Controle de Versões (Anexo A)
- **Tabela 6** – Aprovações (Anexo A)

LISTA DE ABREVIATURAS E SIGLAS

Sigla	Significado
API	Application Programming Interface
EAP	Estrutura Analítica do Projeto
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IPVA	Imposto sobre a Propriedade de Veículos Automotores
JWT	JSON Web Tokens
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language

SUMÁRIO

1 INTRODUÇÃO	6
• 1.1 Justificativa	7
• 1.2 Objetivo Geral	7
• 1.3 Objetivos Específicos	7
• 1.4 Estrutura do Trabalho	7
2 FUNDAMENTAÇÃO TEÓRICA	8
• 2.1 Gestão de Frotas	9
• 2.2 Aplicações Web e Arquitetura Cliente-Servidor	9
• 2.3 Arquitetura REST e API Backend	9
• 2.4 Banco de Dados Relacional e ORM	9
• 2.5 Autenticação e Controle de Acesso	10
• 2.6 Validação de Dados e Qualidade do Software	10
3 METODOLOGIA	10
• 3.1 Abordagem Ágil e Ciclo de Desenvolvimento	10
• 3.2 Versionamento e Modelagem	11
4 DESENVOLVIMENTO	11
• 4.1 Estrutura do Monorepo	11
• 4.2 Backend e Integração	11
• 4.3 Frontend e Usabilidade	11
REFERÊNCIAS	12
ANEXOS	13
• Anexo A – Termo de abertura do projeto	13
• Anexo B – Documento de arquitetura de software	16

• INTRODUÇÃO

A gestão de frotas é uma atividade estratégica para organizações que dependem do uso contínuo de veículos em suas operações, pois envolve o acompanhamento de custos, controle de manutenções, regularização documental e monitoramento do desempenho operacional. Quando esse gerenciamento é realizado por meio de controles manuais ou planilhas isoladas, tornam-se mais frequentes problemas como falhas no registro de informações, dificuldade no acompanhamento de despesas, ausência de previsibilidade orçamentária e riscos relacionados ao vencimento de documentos obrigatórios e à ocorrência de multas.

Nesse contexto, o projeto **Fleet Manager – Sistema de Gestão Inteligente de Frotas** surge como uma proposta de solução tecnológica voltada à centralização e organização das informações operacionais, financeiras e documentais de veículos. O sistema será desenvolvido como uma aplicação web, com o objetivo de oferecer maior eficiência no controle da frota, apoiar a tomada de decisão e reduzir riscos administrativos e legais associados à má gestão desses recursos.

A proposta do Fleet Manager contempla funcionalidades essenciais para a rotina de gestão, como cadastro de veículos, motoristas e usuários, registro de despesas operacionais, controle de manutenções preventivas e corretivas, monitoramento de vencimentos de documentos e geração de indicadores financeiros. Além disso, o sistema contará com mecanismos de autenticação e controle de acesso por perfil de usuário, contribuindo para a segurança e integridade das informações armazenadas.

Dessa forma, o desenvolvimento deste trabalho está inserido no contexto do **Projeto Final Integrador I**, tendo como finalidade formalizar a concepção inicial do sistema, apresentar seus objetivos, delimitar seu escopo e justificar sua relevância. O Fleet Manager busca atender a uma necessidade real de organização e modernização do controle de frotas, propondo uma solução viável, escalável e alinhada às demandas de gestão contemporânea.

1.1 JUSTIFICATIVA

A realização deste trabalho justifica-se pela necessidade de desenvolver uma solução que minimize os problemas decorrentes da gestão de frotas feita de maneira manual ou descentralizada. Em muitos contextos, o uso de planilhas isoladas e controles pouco integrados

dificulta o acompanhamento preciso de despesas, manutenções, multas e vencimentos de documentos, comprometendo a eficiência operacional e aumentando os riscos financeiros e legais.

O Fleet Manager propõe centralizar essas informações em um único sistema web, permitindo maior controle sobre os veículos e seus custos associados. Com isso, espera-se contribuir para a redução de desperdícios, melhoria da organização dos dados, aumento da previsibilidade orçamentária e apoio à tomada de decisão estratégica por parte dos gestores. Assim, o projeto se mostra relevante tanto do ponto de vista acadêmico quanto prático, por abordar um problema real e propor uma solução tecnológica aplicável.

1.2 OBJETIVO GERAL

Desenvolver um sistema web para gestão inteligente de frotas, com foco no controle operacional, financeiro e documental de veículos, promovendo maior previsibilidade de custos, redução de riscos legais e suporte à tomada de decisão estratégica.

1.3 OBJETIVOS ESPECÍFICOS

- Desenvolver funcionalidades para cadastro, edição e gerenciamento de veículos, motoristas e usuários do sistema.
- Permitir o registro de despesas vinculadas aos veículos, contemplando diferentes categorias, como combustível, manutenção, multas, IPVA e seguros.
- Implementar o controle de manutenções preventivas e corretivas, com acompanhamento de seus respectivos status.
- Registrar documentos obrigatórios dos veículos, com monitoramento de datas de vencimento e emissão de alertas.
- Disponibilizar indicadores financeiros básicos, como custo por veículo e evolução mensal das despesas.
- Implementar autenticação de usuários e controle de acesso por perfil, garantindo maior segurança e organização no uso do sistema.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está estruturado de forma a apresentar, inicialmente, os elementos introdutórios do projeto, incluindo a contextualização do tema, a justificativa, o objetivo geral e

os objetivos específicos. Em seguida, no desenvolvimento, serão abordados os aspectos relacionados ao escopo do sistema, aos requisitos funcionais e não funcionais, ao cronograma resumido e às principais partes interessadas envolvidas no projeto.

Por fim, a conclusão apresentará uma síntese dos principais pontos discutidos, destacando a relevância do Fleet Manager como solução para o gerenciamento de frotas e sua contribuição para a organização, controle e apoio estratégico nas operações que envolvem veículos.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento do Fleet Manager, abordando os conceitos de gestão de frotas, as principais tecnologias adotadas no projeto e os padrões de software utilizados na construção do sistema.

2.1 GESTÃO DE FROTAS

A gestão de frotas compreende o conjunto de atividades voltadas ao planejamento, organização, controle e monitoramento dos veículos utilizados por uma organização. Esse processo envolve o acompanhamento de custos operacionais, a manutenção preventiva e corretiva dos veículos, o gerenciamento de documentação obrigatória e o monitoramento do desempenho dos motoristas. Quando realizada de forma manual ou por meio de planilhas isoladas, a gestão de frotas está sujeita a erros humanos, inconsistências nos dados e dificuldades no acompanhamento histórico (FLEURY; WANKE; FIGUEIREDO, 2000).

Sistemas informatizados de gestão de frotas permitem superar essas limitações ao centralizar as informações e automatizar processos críticos, como o envio de alertas sobre vencimentos de documentos, o cálculo de indicadores financeiros e o controle de status de manutenções. A informatização reduz o risco de multas por documentos vencidos, melhora a previsibilidade orçamentária e apoia a tomada de decisão estratégica pelos gestores.

2.2 APLICAÇÕES WEB E ARQUITETURA CLIENTE-SERVIDOR

Os sistemas de informação baseados na web são aplicações acessadas por meio de navegadores, fundamentadas na arquitetura cliente-servidor. Nessa arquitetura, o cliente é responsável pela interface com o usuário (frontend), enquanto o servidor processa as requisições e gerencia os dados (backend). Segundo Sommerville (2011), sistemas baseados na web

permitem acesso ubíquo e facilitam a manutenção centralizada do software, tornando-se uma escolha natural para aplicações corporativas.

Uma categoria relevante de aplicações web é a Single Page Application (SPA), que carrega uma única página HTML e atualiza o conteúdo dinamicamente sem recarregar a página inteira. Para o desenvolvimento da interface (frontend) do Fleet Manager, foi escolhida a biblioteca React, em conjunto com TypeScript, proporcionando uma tipagem estática que previne erros. A construção (build) e execução do ambiente são feitas utilizando o Vite, devido à sua rapidez e eficiência. O estilismo da aplicação é gerido pelo Tailwind CSS, um framework utilitário que permite a criação de interfaces responsivas e modernas de forma ágil. Além disso, o sistema conta com suporte a internacionalização utilizando as bibliotecas react-i18next e i18next.

2.3 ARQUITETURA REST E API BACKEND

REST (Representational State Transfer) é um estilo arquitetural para sistemas distribuídos proposto por Fielding (2000), que utiliza o protocolo HTTP e seus métodos (GET, POST, PUT e DELETE) para comunicação entre cliente e servidor, operando sobre recursos identificados por URLs. Uma API RESTful promove a separação de responsabilidades entre frontend e backend, viabilizando o desenvolvimento independente de cada camada e facilitando a integração com diferentes tipos de clientes.

No contexto do Fleet Manager, o backend foi construído para expor endpoints RESTful utilizando Node.js. A organização em camadas garante a separação de responsabilidades, facilitando a manutenção. O consumo desses endpoints pelo frontend é feito por meio de uma configuração centralizada, gerenciando a injeção do token de autorização e o tratamento das requisições de forma padronizada.

2.4 BANCO DE DADOS RELACIONAL E ORM

O PostgreSQL é um sistema gerenciador de banco de dados relacional de código aberto, reconhecido por sua robustez e conformidade com o padrão SQL. Essas características garantem a integridade referencial dos dados em cenários com múltiplas entidades relacionadas, como veículos, motoristas, despesas e documentos.

Em relação ao armazenamento de dados, o projeto utiliza o Supabase, uma plataforma que provê um banco de dados PostgreSQL totalmente gerenciado, acessado de forma unificada através do Prisma ORM (Object-Relational Mapping). O Prisma facilita a modelagem do banco e as operações de acesso aos dados, atuando como a única interface de comunicação entre a

aplicação e o PostgreSQL. Isso reduz a ocorrência de erros em tempo de execução e eleva a produtividade no desenvolvimento.

2.5 AUTENTICAÇÃO E CONTROLE DE ACESSO

A autenticação é o processo de verificar a identidade de um usuário, enquanto a autorização determina quais recursos esse usuário pode acessar. A arquitetura do sistema utiliza uma solução de autenticação própria (custom auth), abandonando dependências externas como o Auth0.

Essa nova abordagem utiliza JSON Web Tokens (JWT) gerados internamente, onde o token é validado e armazenado de maneira segura (via localStorage no frontend). O hash de senhas é realizado por meio da biblioteca bcryptjs e a manipulação dos tokens é feita via biblioteca jose. Ao efetuar login com email e senha, o sistema emite um token JWT que é enviado em todas as requisições subsequentes no cabeçalho Authorization.

O controle de acesso é implementado segundo o modelo RBAC (Role-Based Access Control). O sistema possui três papéis: ADMIN, com acesso irrestrito; MANAGER, responsável pelo gerenciamento operacional; e OPERATOR, com permissões restritas ao registro de informações e operações do dia a dia.

2.6 VALIDAÇÃO DE DADOS E QUALIDADE DO SOFTWARE

A validação dos dados de entrada é uma prática essencial para garantir a integridade e a segurança das informações. O Zod é uma biblioteca de validação e parsing de esquemas para TypeScript que permite definir, de forma declarativa, a estrutura esperada dos dados recebidos nas requisições HTTP (COLINHACKS, 2024).

Em relação à qualidade do software, os testes automatizados foram implementados com o framework Vitest, que oferece execução rápida e suporte nativo a TypeScript, garantindo que os comportamentos esperados das regras de negócio sejam validados de ponta a ponta.

3 METODOLOGIA

A metodologia adotada para o desenvolvimento do Fleet Manager baseou-se em abordagens ágeis e estruturadas, permitindo iterações rápidas e validação contínua do escopo.

3.1 ABORDAGEM ÁGIL E CICLO DE DESENVOLVIMENTO

O projeto foi dividido em ciclos incrementais (sprints), focando na entrega de conjuntos de funcionalidades específicas a cada etapa. A Sprint 0 foi dedicada ao setup da arquitetura

(monorepo com Turborepo), padronização de código (ESLint, Prettier) e configurações de banco de dados e ORM.

A Sprint 1 contemplou a camada de Autenticação e Acesso, enquanto a Sprint 2 concentrou-se no CRUD e gestão centralizada de Veículos. Sprints subsequentes expandiram o sistema para áreas como gestão de documentos (upload de arquivos), dashboard de indicadores e gerenciamento avançado de usuários.

3.2 VERSIONAMENTO E MODELAGEM

O controle de versão foi realizado através do Git, com o código fonte hospedado no GitHub. A fase de modelagem incluiu a definição de fluxos de acesso, casos de uso e diagramas de entidade-relacionamento (DER), garantindo um planejamento arquitetural consistente antes do desenvolvimento do código.

4 DESENVOLVIMENTO

O desenvolvimento técnico do sistema aplicou os conceitos e tecnologias definidos na fundamentação teórica para entregar uma solução robusta e escalável.

4.1 ESTRUTURA DO MONOREPO

Foi adotada a estrutura de monorepo gerenciada pelo Turborepo, permitindo separar o código em aplicações ('apps', como o backend e o frontend) e pacotes compartilhados ('packages', contendo as configurações comuns de TypeScript e linting). Essa arquitetura garante o reaproveitamento de código e simplifica os pipelines de integração.

4.2 BACKEND E INTEGRAÇÃO

A API backend processa as regras de negócio em rotas protegidas pelo middleware de autenticação (que valida os JWTs via jose). As consultas e mutações de banco de dados são delegadas ao Prisma Client, que interage nativamente com o banco PostgreSQL no Supabase, oferecendo alta performance e segurança.

4.3 FRONTEND E USABILIDADE

No frontend, utilizou-se o React Router DOM para gerenciar as rotas. A comunicação com a API foi centralizada em uma utilidade de 'fetch' configurada para injetar o token dinamicamente nas requisições. As interfaces foram implementadas utilizando o Tailwind CSS para garantir responsividade e uma excelente experiência de usuário (UX), fundamental para gestores e operadores na administração diária da frota.

REFERÊNCIAS

[REACT] META. React Official Documentation. Biblioteca utilizada para a construção da Interface de Usuário (Camada Cliente). Disponível em: <https://react.dev/>. Acesso em: 11 jun. 2026.

[TS] MICROSOFT. TypeScript Documentation. Documentação oficial da linguagem utilizada no ecossistema do projeto. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 11 jun. 2026.

[NODE] OPENJS FOUNDATION. Node.js Official Documentation. Ambiente de execução utilizado na Camada de Serviço REST (Backend). Disponível em: <https://nodejs.org/en/docs/>. Acesso em: 11 jun. 2026.

[PRISMA] PRISMA DATA INC. Prisma Documentation. Ferramenta ORM (Object-Relational Mapping) adotada para o mapeamento e as consultas ao banco de dados relacional. Disponível em: <https://www.prisma.io/docs/>. Acesso em: 11 jun. 2026.

[SUPABASE] SUPABASE. Supabase Documentation. Plataforma de Backend-as-a-Service (BaaS) que provê o banco de dados PostgreSQL totalmente gerenciado. Disponível em: <https://supabase.com/docs>. Acesso em: 11 jun. 2026.

[TAILWIND] TAILWIND LABS. Tailwind CSS Documentation. Framework CSS utilitário utilizado para o desenvolvimento visual do frontend. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 11 jun. 2026.

[TURBOREPO] VERCEL INC. Turborepo Documentation. Ferramenta de construção (build system) de alta performance adotada para o gerenciamento da arquitetura em Monorepo. Disponível em: <https://turbo.build/repo/docs>. Acesso em: 11 jun. 2026.

[VERCEL] VERCEL INC. Vercel Documentation. Plataforma em nuvem escolhida para a hospedagem e a distribuição global dos artefatos estáticos do frontend. Disponível em: <https://vercel.com/docs>. Acesso em: 11 jun. 2026.

[VITE] VITE TEAM. Vite Documentation. Ferramenta de construção e bundler de próxima geração adotada pelo frontend. Disponível em: <https://vitejs.dev/guide/>. Acesso em: 11 jun. 2026.

ANEXO A – Termo de abertura do projeto

Controle de Versões			
Versão	Data	Autor	Notas da Revisão
1.0	23/02/2026	Luiz Eduardo	Versão inicial do documento
1.1	11/06/2026	André Cavalcante	Versão final do documento

1 Objetivos deste documento

Autorizar o início do projeto Fleet Manager, definir seus objetivos, escopo e justificativa, formalizando as informações iniciais necessárias para o desenvolvimento do Projeto Final Integrador I.

2 Identificação do Projeto

Nome do Projeto: Fleet Manager – Sistema de Gestão Inteligente de Frotas

Código do Projeto: FM-PFI-2026

3 Objetivos

Descrição Geral: Desenvolver um sistema web para gestão de frotas, com foco no controle operacional, financeiro e documental de veículos, permitindo maior previsibilidade de custos, redução de riscos legais e apoio à tomada de decisão estratégica.

- Implementar cadastro de veículos, motoristas e usuários;
- Permitir registro de despesas vinculadas aos veículos;
- Controlar manutenções preventivas e corretivas;
- Monitorar vencimento de documentos e multas;
- Gerar indicadores como custo por veículo e evolução mensal de despesas;
- Implementar controle de acesso por perfil de usuário.

4 Escopo do Projeto

Escopo Principal:

O projeto contempla o desenvolvimento de um sistema web para gestão de frotas, incluindo o cadastro e a gestão de veículos e motoristas, bem como o registro de despesas operacionais vinculadas a cada veículo, tais como combustível, peças, serviços, multas, IPVA e seguros.

O sistema permitirá o controle de manutenções preventivas e corretivas, com acompanhamento de seus respectivos status (programada, realizada ou atrasada). Também será implementado o controle de documentos obrigatórios, com alertas automáticos para vencimentos.

Além disso, o sistema disponibilizará um painel (dashboard) com indicadores financeiros básicos, como custo por veículo e evolução mensal de despesas, bem como mecanismos de autenticação e controle de acesso por perfil de usuário.

Exclusões do escopo:

Não fazem parte do escopo deste projeto funcionalidades de rastreamento GPS em tempo real, telemetria avançada, planejamento e otimização de rotas, integração automática com sistemas governamentais (como DETRAN) e desenvolvimento de aplicativo mobile nativo.

5 Justificativa do Projeto

A gestão de frotas realizada por controles manuais ou planilhas isoladas gera falhas no controle financeiro, ausência de previsibilidade orçamentária e riscos relacionados a documentos vencidos e multas.

O Fleet Manager propõe uma solução estruturada para centralizar dados operacionais e financeiros, permitindo melhor controle de custos, redução de desperdícios, maior organização das informações e apoio estratégico à tomada de decisão.

6 Principais requisitos das principais entregas/produtos

Os requisitos abaixo estão relacionados às entregas definidas na EAP do projeto Fleet Manager.

6.1 Requisitos Funcionais (RF)

- RF01 – O sistema deverá permitir o cadastro, edição e exclusão de veículos.
- RF02 – O sistema deverá permitir o cadastro e gerenciamento de motoristas.
- RF03 – O sistema deverá permitir o registro de despesas vinculadas obrigatoriamente a um veículo.
- RF04 – O sistema deverá permitir o registro de diferentes tipos de despesas (combustível, manutenção, multas, IPVA, seguros, entre outros).
- RF05 – O sistema deverá permitir o controle de manutenções preventivas e corretivas.
- RF06 – O sistema deverá registrar documentos obrigatórios com data de vencimento.
- RF07 – O sistema deverá emitir alertas para vencimentos próximos de documentos e manutenções.
- RF08 – O sistema deverá gerar indicadores financeiros básicos, como custo por veículo e evolução mensal de despesas.
- RF09 – O sistema deverá possuir autenticação de usuários.
- RF10 – O sistema deverá controlar níveis de acesso por perfil (Administrador, Gestor e Operador).

6.2 Requisitos Não Funcionais (RNF)

- RNF01 – O sistema deverá ser desenvolvido como aplicação web responsiva.
- RNF02 – O sistema deverá garantir armazenamento seguro dos dados em banco de dados relacional.
- RNF03 – O sistema deverá possuir controle de autenticação e autorização.
- RNF04 – O sistema deverá apresentar tempo de resposta adequado às operações básicas (até 3 segundos em operações comuns).
- RNF05 – O sistema deverá ser desenvolvido utilizando arquitetura cliente-servidor.
- RNF06 – O sistema deverá permitir escalabilidade futura para inclusão de novas funcionalidades.
- RNF07 – O sistema deverá manter integridade e consistência das informações registradas.

7 Cronograma Resumido

Data de Início

01/03/2025

Data de Término

30/06/2025

Fase ou Grupo de Processos	Marcos	Previsão
Iniciação	Projeto aprovado	01/03/2025
Planejamento	Plano de Gerenciamento do Projeto aprovado	15/03/2025
Planejamento	Linhas de Base de Escopo, Prazo e Custos definidas	20/03/2025
Execução	Desenvolvimento do Backend concluído	30/04/2025

Execução	Desenvolvimento do Frontend concluído	20/05/2025
Monitoramento e Controle	MVP validado e testado	10/06/2025
Encerramento	Projeto entregue e apresentado	30/06/2025

8 Partes interessadas do projeto (Stakeholders)

As principais partes interessadas do projeto são apresentadas a seguir:

Empresa / Instituição	Participante	Função / Responsabilidade	Nível de Autoridade
UNIFOR	Professor da Disciplina	Patrocinador acadêmico, responsável pela orientação, acompanhamento e avaliação do projeto	Alto
Projeto Fleet Manager	Luiz Eduardo	Gerente do Projeto e membro da Equipe de Desenvolvimento	Alto
Projeto Fleet Manager	Gregory Jereissati	Gerente do Projeto e membro da Equipe de Desenvolvimento	Alto
Projeto Fleet Manager	André Cavalcante	Gerente do Projeto e membro da Equipe de Desenvolvimento	Alto
Projeto Fleet Manager	Equipe de Desenvolvimento (integrantes do grupo)	Responsável pelo desenvolvimento técnico, modelagem, implementação e documentação do sistema	Médio
Usuários Finais	Gestores de Frota	Validação e utilização estratégica do sistema	Médio
Usuários Finais	Operadores	Registro e atualização das informações operacionais	Baixo

9 Restrições

[Relacione as restrições do projeto, ou seja, limitação aplicável ao projeto, a qual afetará seu desempenho. Limitações reais: orçamento, recursos, tempo de alocação, ... Ex.: Orçamento de R\$1.500.000,00]

10 Riscos

[Descreva os principais riscos do projeto.]

[Principais Riscos: Identifique os riscos mais relevantes que podem afetar o projeto.
Plano de Mitigação: Esboce as estratégias para minimizar esses riscos.]

11 Critérios de Sucesso

[Indicadores de Sucesso: Defina como o sucesso do projeto será medido.

Critérios de Aceitação: Especifique os critérios que devem ser atendidos para que o projeto seja considerado concluído com êxito.]

Aprovações		
Participante	Assinatura	Data
Patrocinador do Projeto		
Gerente do Projeto		

ANEXO B – Termo de abertura do projeto

Documento de Arquitetura de Software

Fleet Manager – Sistema de Gestão Inteligente de Frotas

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

<u>Data</u>	<u>Demanda</u>	<u>Autor</u>	<u>Descrição</u>	<u>Versão</u>
[18/03/2026]	[1]	[André Cavalcante]	Start no documento de arquitetura de software	[1.0]
[11/06/2026]	[2]	[André Cavalcante]	Finalização no documento de arquitetura de software	[2.0]

Sumário

1. INTRODUÇÃO	3
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	4
4. VISÃO DE CASOS DE USO	5
4.1 Casos de Uso significantes para a arquitetura	5
5. VISÃO LÓGICA	6
5.1 Visão Geral – pacotes e camadas	6
6. VISÃO DE IMPLEMENTAÇÃO	8
6.1 Caso de Uso [00X]	8
6.1.1 Diagrama de Classes	8
6.1.2 Diagrama de Sequência	9
7. VISÃO DE IMPLANTAÇÃO	10
8. PROJETO DE BANCO DE DADOS	10
8.1 Modelo Conceitual	10
8.2 Modelo Lógico	10

1. Introdução

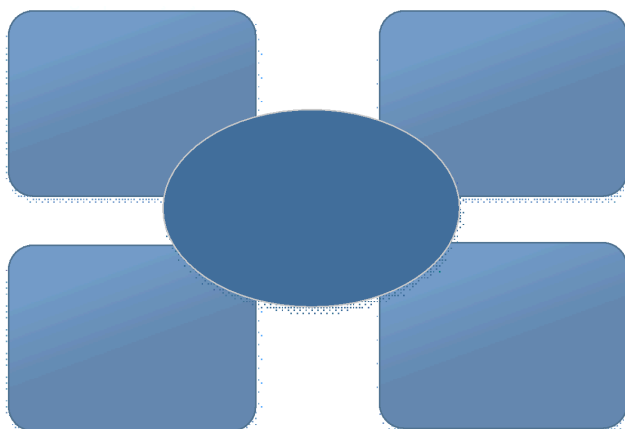


Figura 1 – Arquitetura 4+1

- **1.2 Escopo**

-Com base na necessidade de uma entrega robusta e escalável, a arquitetura deve atender aos seguintes critérios:

-Linguagens e Plataforma: TypeScript em todo o stack. O backend será Node.js e o frontend será React.js.

-Controle de Acesso: O sistema contará com controle de acesso por perfil (RBAC), dividindo os níveis de acesso entre Administrador, Gestor e Operador. A autenticação será realizada por meio de uma solução própria (custom auth) utilizando JWT gerados internamente e validação com bcryptjs e jose.

-Persistência e Cache: Os dados serão persistidos em um banco relacional PostgreSQL totalmente gerenciado pela plataforma Supabase, com acesso via Prisma ORM.

-Cloud e Deployment: A infraestrutura do backend será hospedada em um ambiente Node.js, com o banco de dados no Supabase. O frontend será hospedado na Vercel.

- **1.3 Definições, Acrônimos e Abreviações**

API REST: Application Programming Interface / Representational State Transfer. Arquitetura de software que permite a comunicação e a troca de dados entre o frontend (interface web do usuário) e o backend (servidor do sistema).

SUPABASE: Plataforma que provê o banco de dados PostgreSQL totalmente gerenciado e atua como provedor central para a persistência e armazenamento do projeto.

BI: Business Intelligence (Inteligência de Negócios). Refere-se à transformação de dados brutos (como despesas de manutenção e combustível) em painéis e indicadores estratégicos para os gestores de frota.

CNH: Carteira Nacional de Habilitação. Documento obrigatório dos motoristas que será registrado e monitorado pelo sistema para emissão de alertas automáticos de vencimento.

JWT: JSON Web Token. Padrão de mercado focado em segurança, utilizado para garantir a autenticação e a autorização dos usuários ao fazerem login no sistema.

MVP: Minimum Viable Product (Produto Mínimo Viável). A versão inicial do Fleet Manager, focada em entregar o escopo principal (cadastro, despesas e manutenções) sem complexidades excessivas, permitindo validação rápida.

RBAC: Role-Based Access Control (Controle de Acesso Baseado em Funções). Método de controle de segurança que restringe o acesso ao sistema com base no papel do usuário (Administrador, Gestor ou Operador).

1.4 Referências

[KRU41]: KRUCHTEN, Philippe. The "4+1" View Model of Software Architecture. IEEE Software, Novembro de 1995. Disponível em: <https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>

[TS]: MICROSOFT. TypeScript Documentation. Documentação oficial da linguagem utilizada no ecossistema do projeto. Disponível em: <https://www.typescriptlang.org/docs/>

[REACT]: META. React Official Documentation. Biblioteca utilizada para a construção da Interface de Usuário (Camada Cliente). Disponível em: <https://react.dev/>

[NODE]: OPENJS FOUNDATION. Node.js Official Documentation. Ambiente de execução utilizado na Camada de Serviço REST. Disponível em: <https://nodejs.org/en/docs/>

[POSTGRES]: THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Sistema de banco de dados relacional adotado para persistência. Disponível em: <https://www.postgresql.org/docs/>

[SUPABASE]: SUPABASE. Supabase Documentation. Plataforma de banco de dados gerenciado PostgreSQL utilizada para persistência. Disponível em: <https://supabase.com/docs>

[VERCEL]: VERCEL. Vercel Documentation. Plataforma em nuvem adotada para hospedar o frontend da aplicação web. Disponível em: <https://vercel.com/docs>

[I18N]: I18NEXT. i18next Documentation. Framework de internacionalização adotado para suporte aos idiomas Português e Inglês. Disponível em: <https://www.i18next.com/>

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

<u>Visão</u>	<u>Público</u>	<u>Área</u>	<u>Modelo da MDS</u>
<u>Lógica</u>	<u>Analistas</u>	<u>Realização dos Casos de Uso</u>	<u>Ex: Diagrama de Classes, Diagrama de Sequência, Modelo de Objetos.</u>
<u>Processo</u>	<u>Integradores</u>	<u>Performance, Escalabilidade, Concorrência</u>	<u>Ex: Diagrama de Atividades, Diagrama de Comunicação (focado em Threads/Runtime).</u>
<u>Implementação</u>	<u>Programadores</u>	<u>Componentes de Software</u>	<u>EX: Diagrama de Componentes, Organização de Packages/Módulos.</u>
<u>Implantação</u>	<u>Gerência de Configuração</u>	<u>Nodos físicos</u>	<u>EX: Diagrama de Implantação (Deployment), Topologia de Rede.</u>
<u>Caso de Uso</u>	<u>Todos</u>	<u>Requisitos funcionais</u>	<u>EX: Diagrama de Casos de Uso, Descrições de Casos de Uso (Protótipos).</u>
<u>Dados</u>	<u>Especialistas em dados</u> <u>Administradores de dados</u>	<u>Persistência de dados</u>	<u>EX: Modelo Entidade-Relacionamento (MER), Dicionário de Dados.</u>

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descrever os requisitos de software e restrições que tem um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	TypeScript em todo o stack. O backend será Node.js e o frontend será React.js.
Plataforma	[Especificar o servidor de aplicações utilizado.] Backend em Node.js hospedado em uma plataforma em nuvem compatível; Frontend em React.js (com Vite) hospedado na Vercel.
Segurança	[Especificar a necessidade de segurança e as características básicas da segurança.] Autenticação e autorização próprias (custom auth) via JWT (JSON Web Token) gerado internamente. Senhas são criptografadas e validadas via bcryptjs e jose.
Persistência	[Especificar a necessidade de persistência e qual o mecanismo de persistência que será adotado.] Banco de dados relacional PostgreSQL totalmente gerenciado pela plataforma Supabase, com acesso aos dados unificado pelo Prisma ORM.
Internacionalização (i18n)	[Especificar a necessidade de internacionalização/localização na aplicação.] Suporte a Português (pt-BR) e Inglês (en-US) utilizando a biblioteca i18next, com detecção automática pelo navegador.

Tabela 2 – Exemplo de requisitos e restrições

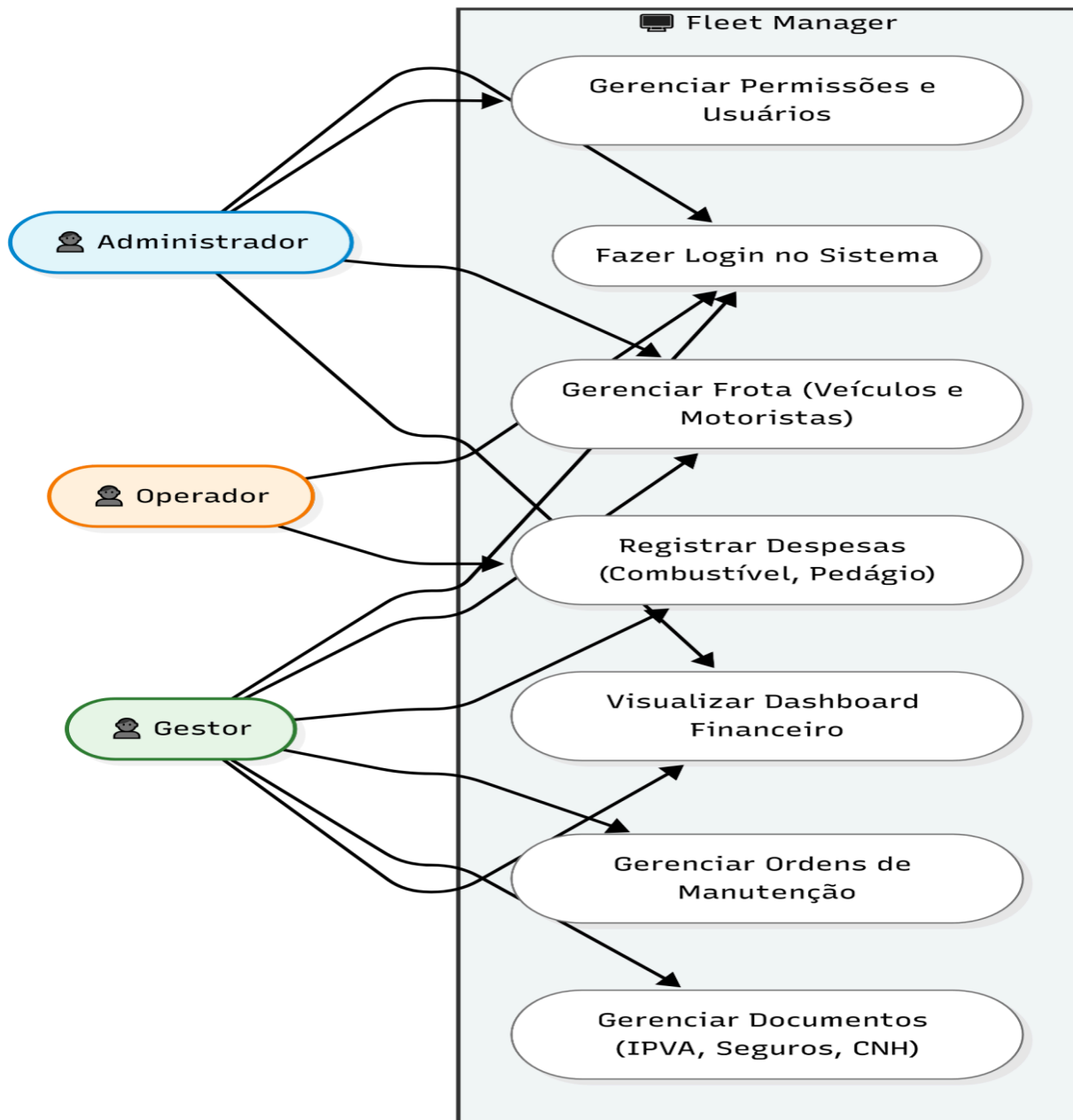
4. VISÃO DE CASOS DE USO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

Lista de casos de uso do sistema:

- 4.1 Casos de Uso significantes para a arquitetura

Figura 1 – Exemplo de Diagrama



com os casos de uso significativos e atores

5. VISÃO LÓGICA

Descrever uma visão lógica da arquitetura. Descrever as classes mais importantes, sua organização em pacotes de serviços e subsistemas, e a organização desses subsistemas em camadas. Também descreve as realizações dos casos de uso mais importantes, por exemplo, aspectos dinâmicos da arquitetura. Diagramas de classes e sequência devem ser incluídos para ilustrar os relacionamentos entre as classes significativas na arquitetura, subsistemas, pacotes e camadas.

5.1 Visão Geral – pacotes e camadas

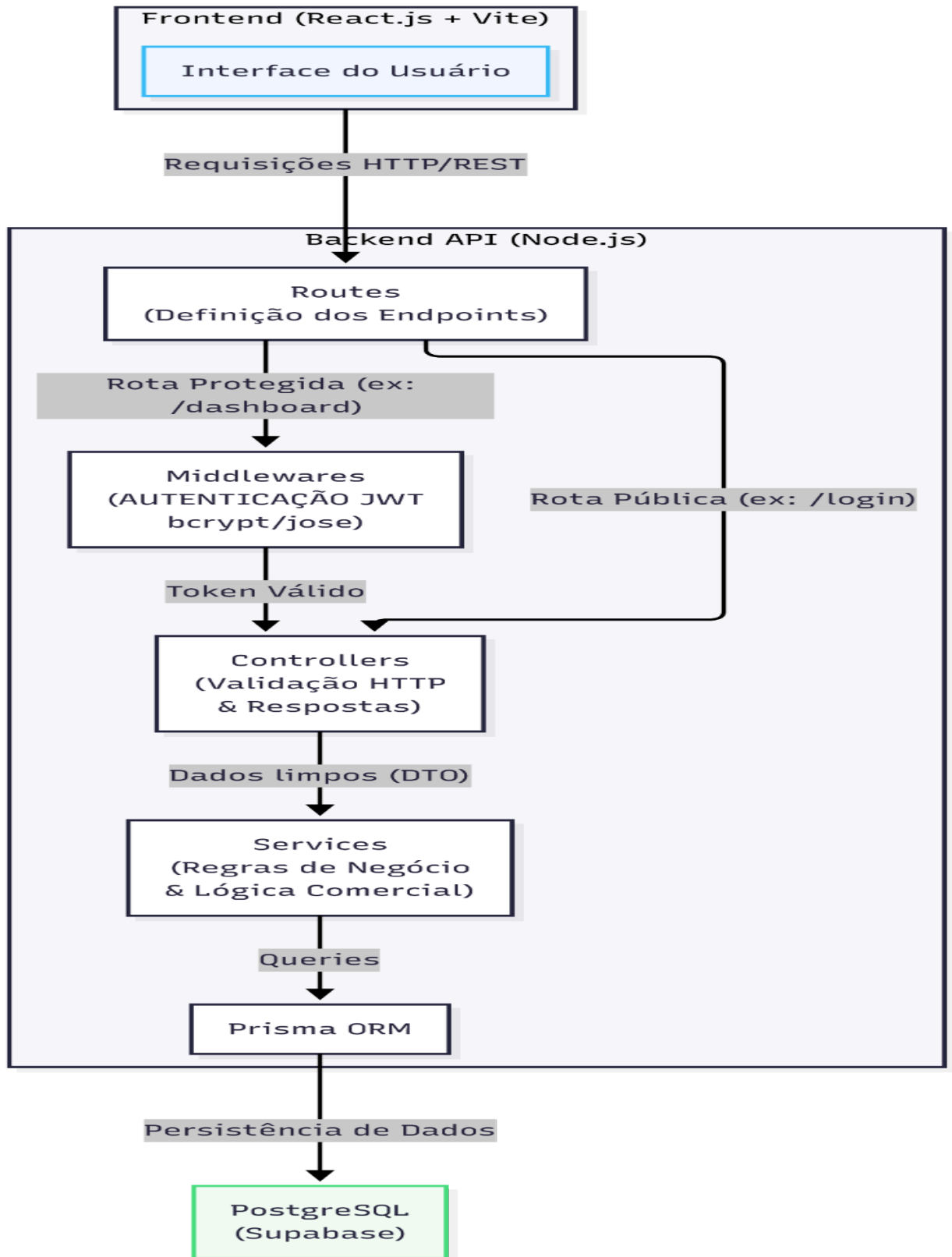


Figura 2 – Exemplo de Diagrama de Camadas da Aplicação

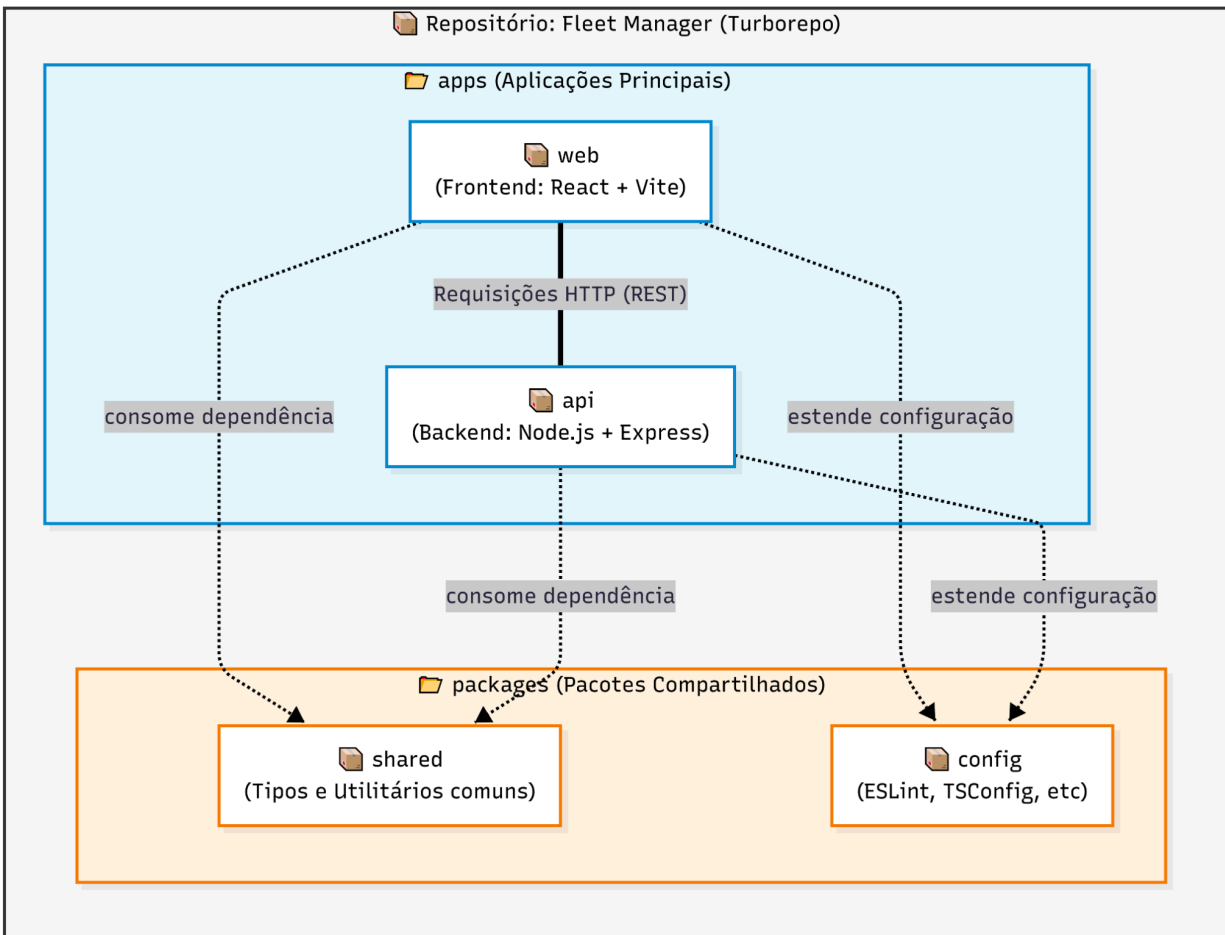


Figura 3 – Exemplo de Diagrama de Pacotes da Aplicação

• 6. VISÃO DE IMPLEMENTAÇÃO

6.1.1 Diagrama de Classes:

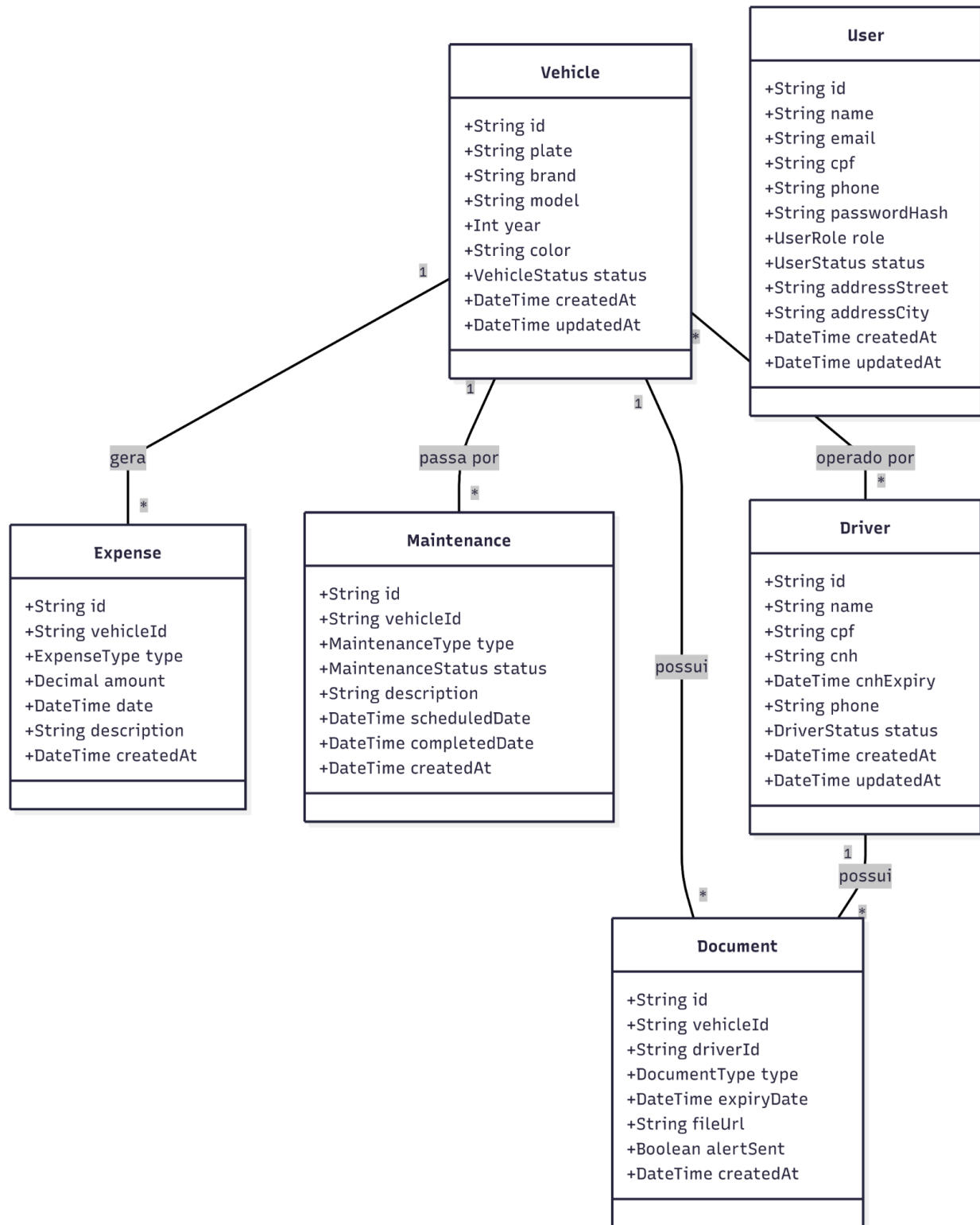
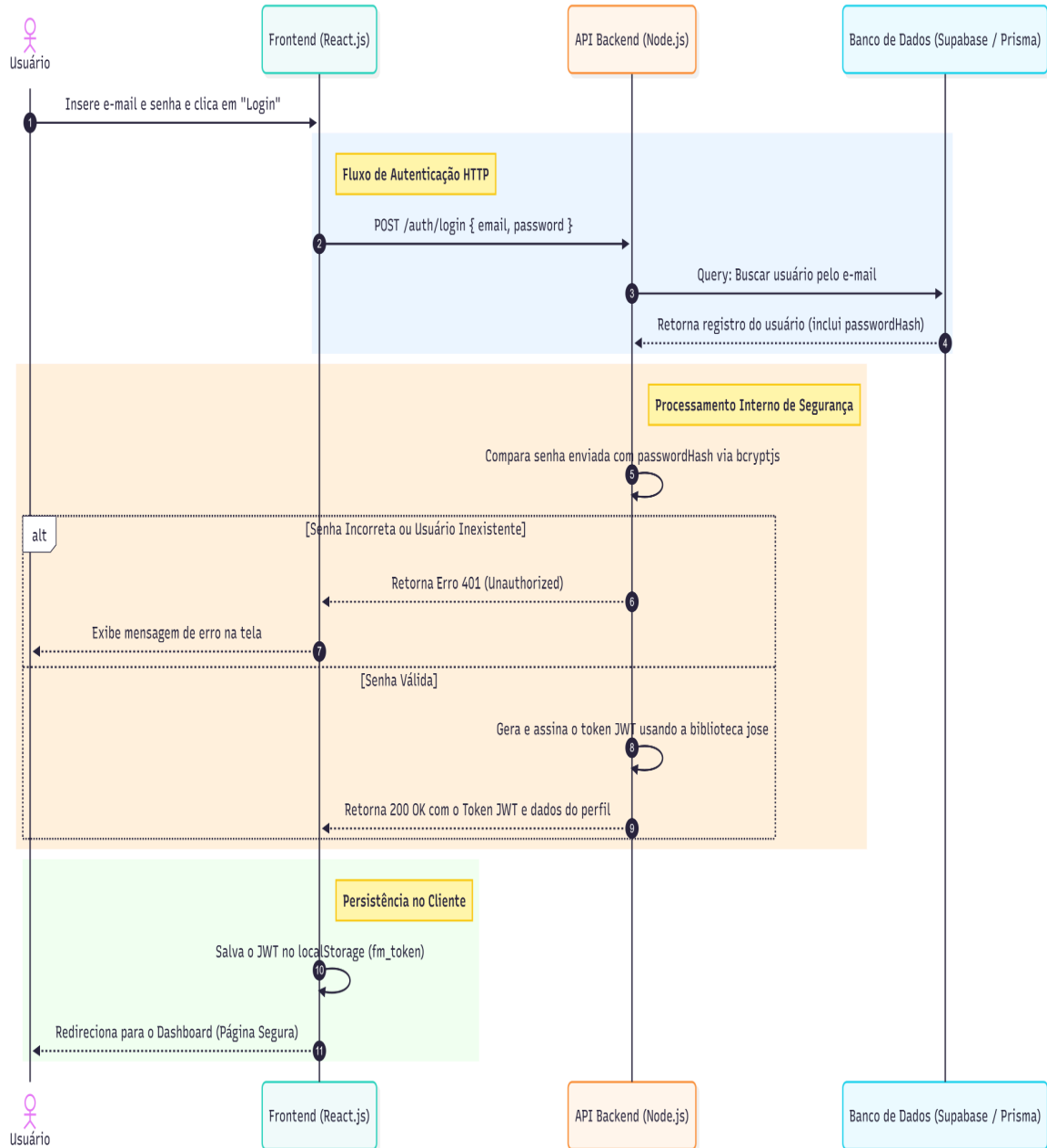


Figura 20 – Exemplo de Diagrama de Classes

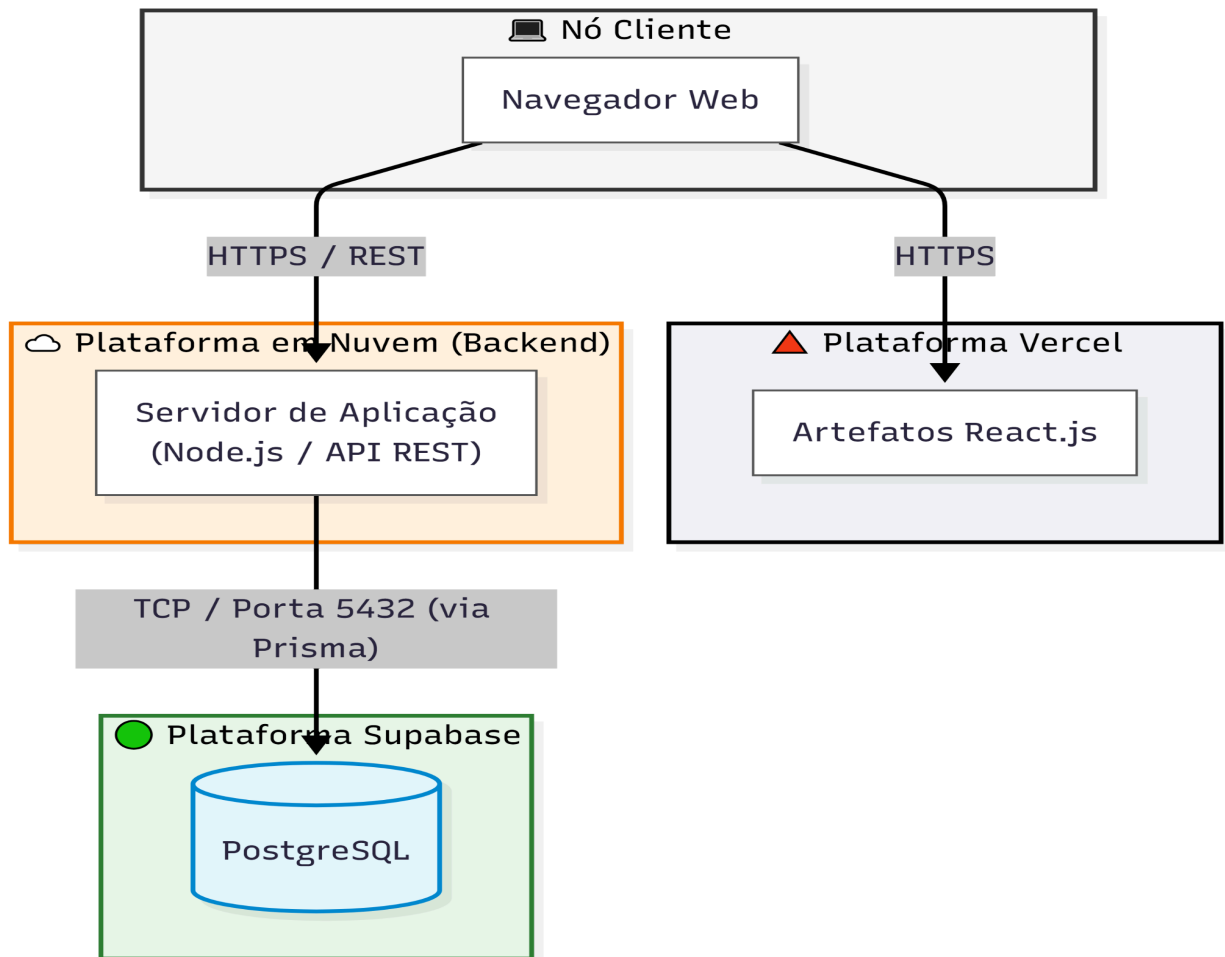
- 6.1.2 Diagrama de Sequência

Figura 20 – Exemplo de Diagrama de Sequência



7. VISÃO DE IMPLANTAÇÃO

- A implantação do sistema Fleet Manager seguirá uma arquitetura baseada em nuvem, separando os artefatos de frontend e backend em provedores especializados para garantir escalabilidade e performance.
- Os principais nodos físicos e configurações incluem:
- **Nó Cliente:** Dispositivos dos usuários finais (operadores e gestores) acessando o sistema através de navegadores web.
- **Nó Frontend (Servidor Web):** Os artefatos estáticos da aplicação visual (React.js) serão hospedados e distribuídos pela infraestrutura da Vercel.
- **Nó Backend (Servidor de Aplicação):** A API REST, construída em Node.js, será hospedada em uma plataforma em nuvem flexível, garantindo a entrega dos endpoints e a comunicação direta com o banco de dados via Prisma.
- **Nó de Persistência (Banco de Dados):** O armazenamento dos dados relacionais será feito em uma instância do PostgreSQL, gerenciada pela plataforma Supabase e acessada via Prisma ORM.



- **8. PROJETO DE BANCO DE DADOS**

- **8.1 Modelo Conceitual**

O modelo conceitual do Fleet Manager foi desenhado para refletir as regras de negócio de gestão de frotas, garantindo a centralização e a rastreabilidade das operações. As principais entidades identificadas para o MVP e seus relacionamentos são:

Veículo: É a entidade central do sistema. Representa o ativo físico da empresa.

Motorista: Representa o colaborador responsável por conduzir os veículos.

Relacionamento: Um Motorista pode operar diversos Veículos ao longo do tempo.

Despesa: Representa qualquer custo financeiro gerado pela operação.

Relacionamento: Toda Despesa deve estar obrigatoriamente vinculada a um único Veículo, permitindo o cálculo do custo por ativo (BI).

Manutenção: Representa as ordens de serviço preventivas e corretivas.

Relacionamento: Uma Manutenção pertence a um Veículo específico.

Documento: Representa a documentação legal e obrigatória (IPVA, Seguros, etc.).

Relacionamento: Um Documento é vinculado a um Veículo para a geração de alertas de vencimento.

- 8.2 Modelo Lógico

